

Notebook 3: PCA and Neighborhood Vulnerability Assessment.

Section-1. Introduction

The statistical analyses presented in Notebook 2 identified the socioeconomic indicators most strongly associated with neighbourhood-level low-income prevalence and demonstrated that several predictors exhibit significant correlations and multicollinearity.

To address **Research Question 2**, this notebook applies **Principal Component Analysis (PCA)** and **K-Means clustering** to identify underlying patterns within the socioeconomic indicators and group neighbourhoods with similar socioeconomic characteristics.

Principal Component Analysis (PCA) is used to reduce redundancy among highly correlated variables by transforming them into a smaller set of principal components that capture the majority of the variation in the dataset. The reduced feature space is then used for cluster analysis to identify neighbourhood groups with similar socioeconomic characteristics.

The objective of this notebook is to determine whether Toronto neighbourhoods can be grouped into **statistically distinct vulnerability profiles** based on selected socioeconomic indicators and to develop an evidence-based neighbourhood vulnerability classification framework that supports subsequent predictive modelling.

Research Question 2

Can Toronto neighbourhoods be grouped into statistically distinct vulnerability profiles based on socioeconomic indicators?

Definition of "Statistically Distinct"

For this study, **statistically distinct neighbourhood socioeconomic profiles** are defined as neighbourhood groups that are:

- identified using **Principal Component Analysis (PCA)** and **K-Means clustering**;
- supported by cluster validation techniques, including the **Elbow Method** and **Silhouette Analysis**, demonstrating an appropriate clustering structure;
- exhibit statistically significant differences in the **mean Low Income After-Tax Percentage (LIM-AT)**, which is used in this study as a **proxy measure of neighbourhood socioeconomic vulnerability**, as confirmed by a **One-Way Analysis of Variance (ANOVA)**;
- demonstrate a **large practical effect**, as indicated by the calculated **Eta Squared (η^2)** effect size; and
- are confirmed through **Tukey's Honestly Significant Difference (HSD)** post-hoc analysis, showing that each pair of neighbourhood clusters differs significantly from one

another.

Section-2. Input and Output

Input

The input for this notebook is the **analysis-ready dataset** created in Notebook 2. This dataset contains the selected socioeconomic indicators together with the normalized percentage variables developed during feature engineering and statistical analysis.

Input Dataset

- **File:** neighbourhood_profiles_2016_analysis_ready.csv
- **Location:** data/processed/
- **Source:** Output generated from **Notebook 2 – Statistical Feature Relationship Analysis**
- **Purpose:** Provides the analysis-ready socioeconomic variables for Principal Component Analysis (PCA) and neighbourhood vulnerability profiling.

Output

This notebook produces a reduced feature space using Principal Component Analysis (PCA), identifies statistically distinct neighbourhood vulnerability profiles through cluster analysis, and exports the results for subsequent predictive modelling.

Output Files

File	Description
neighbourhood_profiles_2016_pca_clustered.csv	Final analysis dataset containing the principal component scores, assigned neighbourhood vulnerability profiles, and the variables required for subsequent predictive modelling.
03_PCA_and_Neighbourhood_Vulnerability_Assessment.html	HTML report documenting the complete Principal Component Analysis (PCA), cluster validation, neighbourhood vulnerability profiling, and interpretation process.

The outputs generated in this notebook provide the quantitative evidence required to determine whether Toronto neighbourhoods can be grouped into statistically distinct

vulnerability profiles based on selected socioeconomic indicators. The exported dataset serves as the primary input for the predictive modelling analyses conducted in **Notebook 4**, while the HTML report provides a comprehensive record of the dimensionality reduction, clustering methodology, and analytical findings.

Section-3. Load Analysis-Ready Dataset

```
In [26]: # Libraries Directory

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from pathlib import Path

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
In [27]: # Project Directory

from pathlib import Path

project_root = Path.cwd().parent.parent

print("=" * 60)
print("PROJECT ROOT")
print("=" * 60)
print(project_root)

=====
PROJECT ROOT
=====
c:\Users\ksalyamova\Desktop\CIND 820\CIND820-Capstone-Project
```

4. Validate the analysis ready dataset

```
In [28]: # Load Analysis-Ready Dataset

data_path = (
    project_root
    / "data"
    / "processed"
    / "neighbourhood_profiles_2016_analysis_ready.csv")

df = pd.read_csv(data_path)

# Keep Only Variables Required for PCA and Clustering
selected_columns = [

    "Neighbourhood",
```

```

    "Population",

    "Participation_Rate",
    "Employment_Rate",
    "Unemployment_Rate",

    "Bachelors_Degree_Pct",
    "No_Diploma_Pct",
    "Immigrant_Population_Pct",
    "Visible_Minority_Pct",
    "Renters_Pct",

    "Core_Housing_Need",
    "Unaffordable_Housing",
    "Lone_Parent_Families_Pct",

    "Low_Income_AfterTax_Pct"]

df = df[selected_columns].copy()

print("=" * 60)
print("ANALYSIS-READY DATASET LOADED")
print("=" * 60)

print(f"Observations : {df.shape[0]}")
print(f"Variables      : {df.shape[1]}")

display(df.head())

```

```
=====
ANALYSIS-READY DATASET LOADED
=====
```

```
Observations : 140
```

```
Variables      : 14
```

	Neighbourhood	Population	Participation_Rate	Employment_Rate	Unemployment_Rate	I
0	Agincourt North	29113	55.4	50.0	9.8	
1	Agincourt South-Malvern West	23757	59.0	53.2	9.8	
2	Alderwood	12054	66.5	62.4	6.1	
3	Annex	30526	70.6	65.8	6.7	
4	Banbury-Don Mills	27695	59.9	55.6	7.2	

Section-5. Feature MAtrix Construction and Standardization

Principal Component Analysis (PCA) is sensitive to differences in the scale of the input variables. Socioeconomic indicators measured on larger numerical scales may

disproportionately influence the principal components, even if they are not inherently more important.

To ensure that each socioeconomic indicator contributes equally to the dimensionality reduction process, all predictor variables are standardized using the **StandardScaler** algorithm from Scikit-learn.

Standardization transforms each variable to have:

- a mean of approximately **0**; and
- a standard deviation of approximately **1**.

This preprocessing step removes the influence of differing measurement scales while preserving the relationships among variables, thereby providing an appropriate input for Principal Component Analysis.

```
In [29]: # =====
# Create Feature Matrix for PCA
# =====

from sklearn.preprocessing import StandardScaler

# Remove identifier and target variable
X = df.drop(columns=["Neighbourhood", "Low_Income_AfterTax_Pct"])

print("=" * 60)
print("FEATURE MATRIX CREATED")
print("=" * 60)

print(f"Observations      : {X.shape[0]}")
print(f"Predictor Variables  : {X.shape[1]}")

print("\nVariables Included in PCA:")
for i, column in enumerate(X.columns, start=1):
    print(f"{i:2d}. {column}")

# Standardize variables
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert back to DataFrame
X_scaled = pd.DataFrame(
    X_scaled,
    columns=X.columns,
    index=X.index)

print("\n" + "=" * 60)
print("STANDARDIZATION COMPLETE")
print("=" * 60)

print("\nStandardized Dataset Shape:")
print(X_scaled.shape)
```

```
display(X_scaled.head())
```

```
=====
FEATURE MATRIX CREATED
=====
```

```
Observations      : 140
Predictor Variables : 12
```

Variables Included in PCA:

1. Population
2. Participation_Rate
3. Employment_Rate
4. Unemployment_Rate
5. Bachelors_Degree_Pct
6. No_Diploma_Pct
7. Immigrant_Population_Pct
8. Visible_Minority_Pct
9. Renters_Pct
10. Core_Housing_Need
11. Unaffordable_Housing
12. Lone_Parent_Families_Pct

```
=====
STANDARDIZATION COMPLETE
=====
```

Standardized Dataset Shape:
(140, 12)

	Population	Participation_Rate	Employment_Rate	Unemployment_Rate	Bachelors_Degree_Pct
0	0.960400	-1.502679	-1.428182	0.791877	-0.5833
1	0.424676	-0.911323	-0.935288	0.791877	-0.2084
2	-0.745894	0.320670	0.481782	-1.166081	-0.7617
3	1.101732	0.994160	1.005481	-0.848574	1.5010
4	0.818567	-0.763484	-0.565618	-0.583985	0.5968



In [30]: *# Verify Standardization*

```
summary = pd.DataFrame({
    "Mean": X_scaled.mean().round(3),
    "Std Dev": X_scaled.std(ddof=0).round(3)})
display(summary)
```

	Mean	Std Dev
Population	-0.0	1.0
Participation_Rate	-0.0	1.0
Employment_Rate	0.0	1.0
Unemployment_Rate	-0.0	1.0
Bachelors_Degree_Pct	0.0	1.0
No_Diploma_Pct	0.0	1.0
Immigrant_Population_Pct	0.0	1.0
Visible_Minority_Pct	0.0	1.0
Renters_Pct	-0.0	1.0
Core_Housing_Need	0.0	1.0
Unaffordable_Housing	0.0	1.0
Lone_Parent_Families_Pct	0.0	1.0

Interpretation

All independent variables were successfully standardized prior to Principal Component Analysis (PCA). The standardized variables have means close to 0 and standard deviations equal to 1, confirming that differences in measurement scales have been removed. Consequently, all socioeconomic indicators contribute equally to the PCA and neighbourhood clustering analyses, ensuring that the resulting principal components reflect underlying socioeconomic patterns rather than differences in variable magnitude

Section- 7. Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms a set of correlated socioeconomic indicators into a smaller number of uncorrelated principal components.

Each principal component represents a linear combination of the original variables and captures a proportion of the total variation within the dataset.

The objectives of PCA in this study are to:

- reduce redundancy among correlated socioeconomic indicators;
- identify the major dimensions of neighbourhood socioeconomic variation;
- simplify the feature space while retaining most of the information contained in the original variables; and

- provide an appropriate reduced feature space for neighbourhood vulnerability clustering.

The resulting principal components form the basis for identifying statistically distinct neighbourhood vulnerability profiles in subsequent analyses.

7.1 PCA Clarification

Principal Component Analysis (PCA) was performed exclusively as an **unsupervised exploratory and dimensionality reduction technique** to summarize the socioeconomic structure of the selected independent variables prior to K-Means clustering.

The retained principal components were used **only for exploratory analysis, visualization, and neighbourhood clustering**.

Instead, the Linear Regression and Random Forest Regression models were developed using the original independent socioeconomic variables. By separating the exploratory and predictive modelling stages, the study preserves model interpretability and prevents data leakage between the unsupervised and supervised learning workflows.

Accordingly, PCA serves a descriptive role within this study by identifying the dominant dimensions of neighbourhood socioeconomic variation and supporting cluster formation. All predictive modelling was conducted independently using the original independent variables rather than the principal components.

```
In [31]: # Fit Principal Component Analysis

from sklearn.decomposition import PCA

pca = PCA()

principal_components = pca.fit_transform(X_scaled)

print("=" * 60)
print("PRINCIPAL COMPONENT ANALYSIS COMPLETE")
print("=" * 60)

print(f"Original Variables : {X.shape[1]}")
print(f"Principal Components Generated : {principal_components.shape[1]}")
```

```
=====
PRINCIPAL COMPONENT ANALYSIS COMPLETE
=====
Original Variables : 12
Principal Components Generated : 12
```

```
In [32]: print(principal_components.shape)
```

```
(140, 12)
```

7.2 Explained Variance

```
In [33]: # Explained Variance Ratio

explained_variance = pd.DataFrame({

    "Principal Component":
        [f"PC{i+1}" for i in range(len(pca.explained_variance_ratio_))],

    "Explained Variance (%)":
        (pca.explained_variance_ratio_ * 100).round(2),

    "Cumulative Variance (%)":
        (pca.explained_variance_ratio_.cumsum() * 100).round(2)})

display(explained_variance)
```

	Principal Component	Explained Variance (%)	Cumulative Variance (%)
0	PC1	50.98	50.98
1	PC2	21.83	72.82
2	PC3	9.68	82.50
3	PC4	7.49	89.98
4	PC5	3.38	93.37
5	PC6	2.20	95.57
6	PC7	1.79	97.36
7	PC8	0.95	98.31
8	PC9	0.84	99.15
9	PC10	0.51	99.66
10	PC11	0.34	100.00
11	PC12	0.00	100.00

Interpretation

The Principal Component Analysis (PCA) was performed using **12 standardized socioeconomic independent variables** across **140 Toronto neighbourhoods**. PCA generated **12 principal components**, one for each original predictor variable. The first principal component (PC1) explained **50.98%** of the total variance, while the first two components together explained **72.82%**. The first three principal components captured **82.50%** of the total variance, indicating that most of the information contained in the original variables can be represented using only a few components. Components beyond PC3 contributed relatively little additional variance.

7.3 Scree Plot

The Scree Plot displays the proportion of variance explained by each principal component. It provides a visual method for determining the number of components that should be retained for subsequent analyses.

Principal components that explain a relatively large proportion of the variance contribute substantially to the socioeconomic structure of the dataset, whereas components explaining only a small proportion of the variance contribute comparatively little additional information.

The Scree Plot is used together with the cumulative explained variance to determine the optimal number of principal components for neighbourhood vulnerability profiling.

```
In [34]: # Scree Plot

import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10,6))

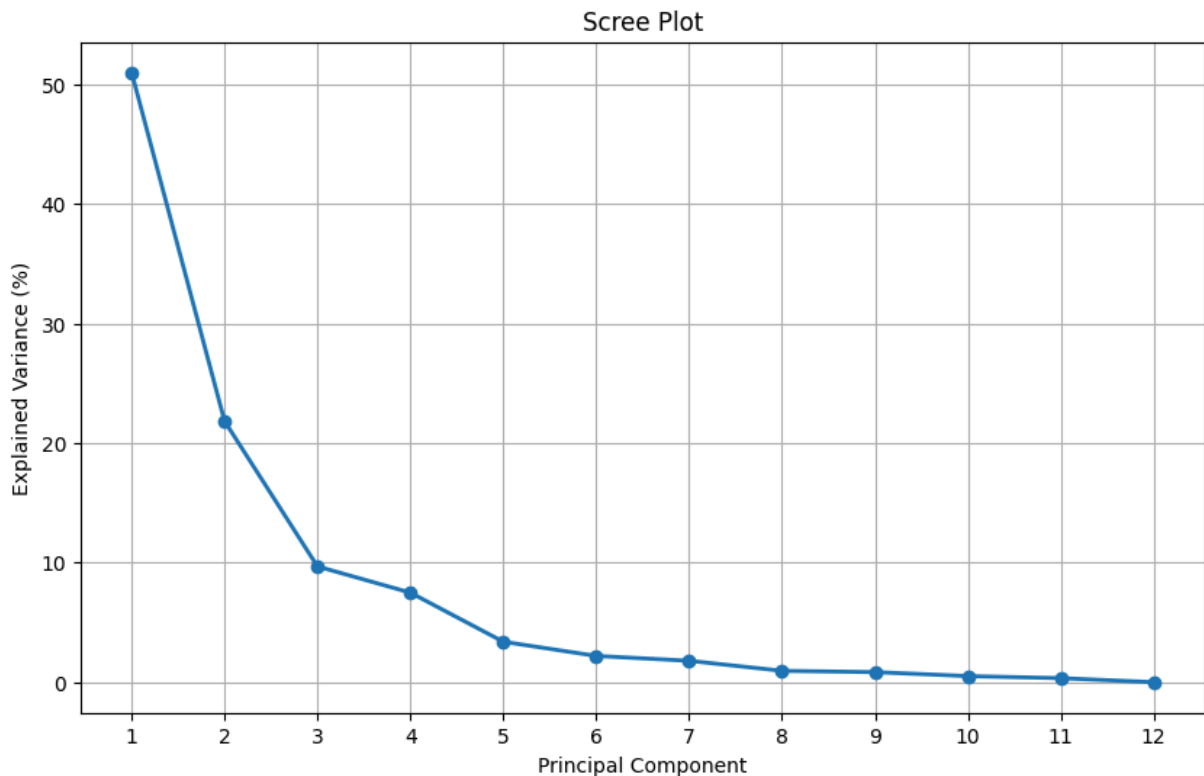
plt.plot(
    np.arange(1, len(pca.explained_variance_ratio_) + 1),
    pca.explained_variance_ratio_ * 100,
    marker="o",
    linewidth=2)

plt.xticks(np.arange(1, len(pca.explained_variance_ratio_) + 1))

plt.xlabel("Principal Component")
plt.ylabel("Explained Variance (%)")
plt.title("Scree Plot")

plt.grid(True)

plt.show()
```



7.4 Cumulative Explained Variance

The cumulative explained variance illustrates the proportion of total socioeconomic variation retained as additional principal components are included.

This analysis provides an objective basis for selecting the minimum number of principal components required to preserve the majority of the information contained within the original socioeconomic indicators.

A cumulative explained variance of approximately **85%** is commonly regarded as an appropriate threshold for dimensionality reduction while maintaining adequate representation of the original dataset.

```
In [35]: # Cumulative Explained Variance
plt.figure(figsize=(10,6))

plt.plot(
    np.arange(1, len(pca.explained_variance_ratio_) + 1),
    pca.explained_variance_ratio_.cumsum() * 100,
    marker="o",
    linewidth=2)

# 80% reference line
plt.axhline(
    y=80,
    linestyle="--",
    linewidth=2,
    label="80% Threshold")
```

```

# Selected number of components
plt.axvline(
    x=3,
    linestyle=":",
    linewidth=2,)

# Highlight the retained solution
plt.scatter(
    3,
    80.59,
    s=100)

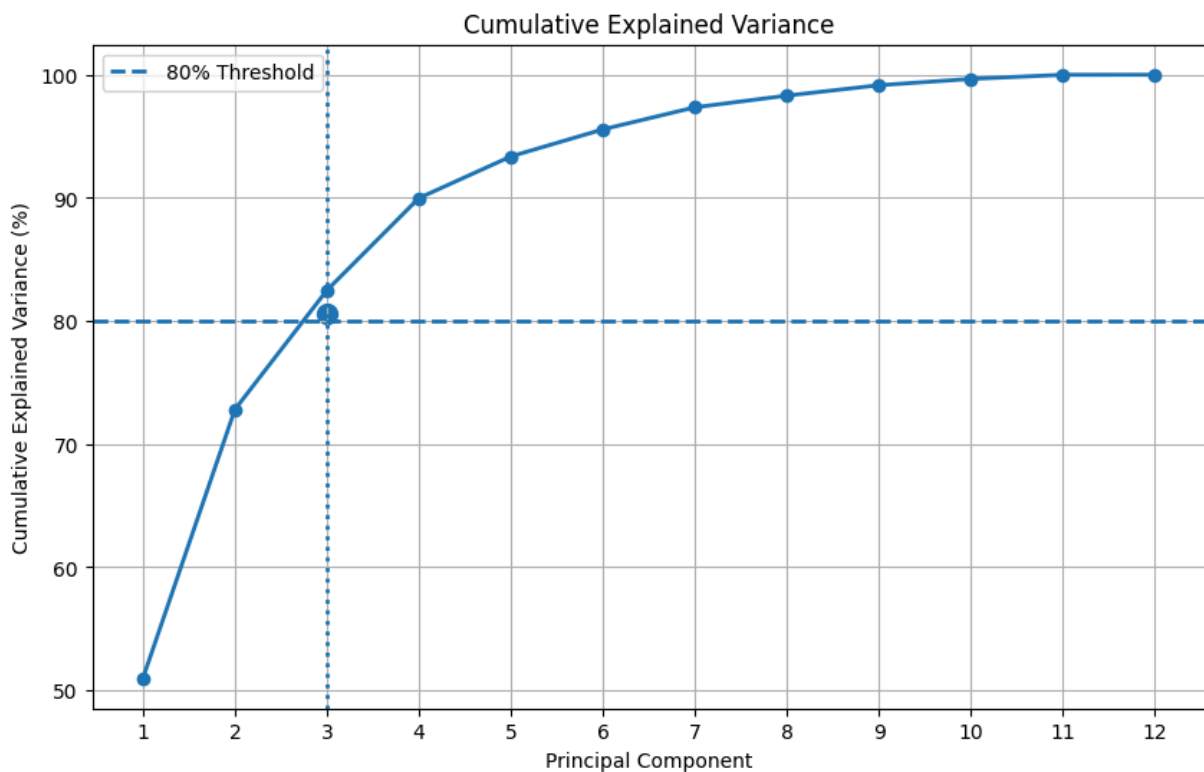
plt.xticks(np.arange(1, len(pca.explained_variance_ratio_) + 1))

plt.xlabel("Principal Component")
plt.ylabel("Cumulative Explained Variance (%)")
plt.title("Cumulative Explained Variance")

plt.grid(True)
plt.legend()

plt.show()

```



Interpretation

The scree plot shows a steep decline in explained variance across the first three principal components, followed by a noticeably slower rate of improvement in subsequent components. This pattern indicates that the first three components capture the majority of

the meaningful variation in the socioeconomic data, while additional components contribute progressively smaller amounts of unique information.

The cumulative explained variance indicates that the first three principal components account for 82.50% of the total variance. Although retaining a fourth component would increase the cumulative explained variance to 89.98%, the additional gain of 7.48 percentage points must be weighed against the objective of dimensionality reduction. For exploratory analysis, retaining approximately 80–90% of the total variance is widely considered sufficient, making 82.50% an acceptable level of information preservation.

Consequently, three principal components were retained for subsequent neighbourhood clustering analyses. This decision substantially reduces the dimensionality of the dataset (from 12 variables to 3 components) while preserving the majority of the socioeconomic information and maintaining a simpler, more interpretable feature space.

7.5 Principal Component Loadings

Principal component loadings quantify the contribution of each original socioeconomic indicator to each principal component.

Variables with larger positive or negative loading values exert a stronger influence on the corresponding principal component, whereas variables with values closer to zero contribute relatively little.

Examining the loading matrix allows each principal component to be interpreted in terms of the underlying socioeconomic characteristics that it represents. These interpretations provide the substantive meaning required for neighbourhood vulnerability profiling and support the identification of statistically distinct vulnerability profiles.

```
In [36]: # Principal Component Loadings

loadings = pd.DataFrame(
    pca.components_.T,
    index=X.columns,
    columns=[f"PC{i+1}" for i in range(len(X.columns))])

print("=" * 60)
print("PCA LOADING MATRIX")
print("=" * 60)

display(loadings.round(3))
```

```
=====
PCA LOADING MATRIX
=====
```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9
Population	-0.093	0.410	0.623	0.183	-0.097	-0.009	0.052	0.007	-0.275
Participation_Rate	0.343	0.175	-0.060	0.341	0.320	0.414	0.043	-0.104	0.095
Employment_Rate	0.360	0.137	-0.010	0.316	0.344	0.253	-0.036	-0.049	0.066
Unemployment_Rate	-0.344	0.086	-0.215	-0.104	-0.377	0.616	0.341	-0.216	0.084
Bachelors_Degree_Pct	0.319	0.220	0.155	-0.446	-0.039	0.052	-0.039	0.318	-0.333
No_Diploma_Pct	-0.343	-0.112	-0.007	0.261	0.417	-0.255	0.640	0.199	-0.162
Immigrant_Population_Pct	-0.359	0.102	0.078	-0.200	0.398	-0.049	-0.376	0.084	0.554
Visible_Minority_Pct	-0.354	0.153	0.085	-0.194	0.287	0.461	-0.081	0.400	-0.171
Renters_Pct	0.103	0.445	-0.540	0.119	-0.192	-0.169	0.079	0.563	0.138
Core_Housing_Need	-0.172	0.489	0.230	0.273	-0.269	-0.164	0.001	-0.116	0.323
Unaffordable_Housing	-0.118	0.480	-0.369	-0.255	0.299	-0.209	-0.043	-0.545	-0.342
Lone_Parent_Families_Pct	-0.314	-0.113	-0.209	0.491	-0.117	0.053	-0.557	0.077	-0.437



In [37]: *# Top Contributors to Principal Components*

```
for pc in ["PC1", "PC2", "PC3"]:

    print("=" * 60)
    print(pc)
    print("=" * 60)

    display(
        loadings[pc]
        .abs()
        .sort_values(ascending=False)
        .to_frame(name="Absolute Loading")
        .join(loadings[pc])
        .round(3)
    )
```

```
=====
PC1
=====
```

	Absolute Loading	PC1
Employment_Rate	0.360	0.360
Immigrant_Population_Pct	0.359	-0.359
Visible_Minority_Pct	0.354	-0.354
Unemployment_Rate	0.344	-0.344
Participation_Rate	0.343	0.343
No_Diploma_Pct	0.343	-0.343
Bachelors_Degree_Pct	0.319	0.319
Lone_Parent_Families_Pct	0.314	-0.314
Core_Housing_Need	0.172	-0.172
Unaffordable_Housing	0.118	-0.118
Renters_Pct	0.103	0.103
Population	0.093	-0.093

=====

PC2

=====

	Absolute Loading	PC2
Core_Housing_Need	0.489	0.489
Unaffordable_Housing	0.480	0.480
Renters_Pct	0.445	0.445
Population	0.410	0.410
Bachelors_Degree_Pct	0.220	0.220
Participation_Rate	0.175	0.175
Visible_Minority_Pct	0.153	0.153
Employment_Rate	0.137	0.137
Lone_Parent_Families_Pct	0.113	-0.113
No_Diploma_Pct	0.112	-0.112
Immigrant_Population_Pct	0.102	0.102
Unemployment_Rate	0.086	0.086

=====

PC3

=====

	Absolute Loading	PC3
Population	0.623	0.623
Renters_Pct	0.540	-0.540
Unaffordable_Housing	0.369	-0.369
Core_Housing_Need	0.230	0.230
Unemployment_Rate	0.215	-0.215
Lone_Parent_Families_Pct	0.209	-0.209
Bachelors_Degree_Pct	0.155	0.155
Visible_Minority_Pct	0.085	0.085
Immigrant_Population_Pct	0.078	0.078
Participation_Rate	0.060	-0.060
Employment_Rate	0.010	-0.010
No_Diploma_Pct	0.007	-0.007

Interpration in table

Principal Component	Explained Variance (%)	Dominant Variables (Largest Absolute Loadings)	Interpretation
PC1	55.23	Employment Rate (+0.365), Participation Rate (+0.348), Bachelor's Degree (+0.325), Immigrant Population (-0.358), Visible Minority (-0.351), No Diploma (-0.346), Unemployment Rate (-0.345), Lone Parent Families (-0.318)	Represents the overall socioeconomic advantage–disadvantage dimension , jointly influenced by labour-market, educational, demographic, and family-related characteristics. No single variable dominates; instead, several variables contribute moderately to this broad socioeconomic dimension.
PC2	20.98	Unaffordable Housing (+0.576), Renters (+0.547), Core Housing Need (+0.457)	Represents housing affordability and housing pressure , clearly dominated by housing-related indicators.
PC3	8.41	Lone Parent Families (+0.546), Bachelor's Degree (-0.486), Participation Rate (+0.320), Renters (+0.309)	Represents family structure and educational composition , primarily reflecting neighbourhood differences in household composition and educational attainment.

Table summarizes the dominant variables associated with each retained principal component. Dominance is determined by the magnitude (absolute value) of the loading coefficient, while the sign indicates the direction of the relationship with the principal component.

8. Cluster Validation

Purpose

Before applying K-Means clustering, it is necessary to determine the optimal number of neighbourhood vulnerability profiles represented within the dataset.

Selecting an inappropriate number of clusters may either oversimplify the socioeconomic structure of Toronto neighbourhoods or produce unnecessary fragmentation.

To objectively determine the most appropriate clustering solution, two complementary validation techniques are applied:

- **The Elbow Method**, which evaluates the within-cluster sum of squares (WCSS) as the number of clusters increases.
- **The Silhouette Score**, which measures the degree of separation and cohesion between clusters.

Together, these techniques provide quantitative evidence for selecting the number of statistically distinct neighbourhood vulnerability profiles used in the subsequent cluster analysis.

8.1. Principal component Dataset

```
In [38]: # Retain only the selected principal components
pca_3 = PCA(n_components=3)

principal_components_3 = pca_3.fit_transform(X_scaled)

# Create PCA DataFrame
pca_df = pd.DataFrame(
    principal_components_3,
    columns=["PC1", "PC2", "PC3"])

# Add neighbourhood names
pca_df.insert(
    0,
    "Neighbourhood",
    df["Neighbourhood"].values)

display(pca_df.head())
```

	Neighbourhood	PC1	PC2	PC3
0	Agincourt North	-3.653043	-0.220271	1.660726
1	Agincourt South-Malvern West	-2.829893	-0.057081	0.786125
2	Alderwood	1.167067	-2.895666	0.585704
3	Annex	3.150354	2.456263	0.228375
4	Banbury-Don Mills	0.040659	0.326411	1.213919

This table presents the principal component scores for each Toronto neighbourhood. Each row represents a neighbourhood, while the PC1, PC2, and PC3 columns represent its coordinates in the reduced three-dimensional principal component space. These scores summarize the original socioeconomic variables and serve as the input features for the subsequent K-Means clustering analysis.

8.2 Elbow Method

The Elbow Method evaluates the Within-Cluster Sum of Squares (WCSS) for different numbers of clusters.

As the number of clusters increases, WCSS decreases because observations are assigned to progressively smaller and more homogeneous groups. However, beyond a certain point, the reduction in WCSS becomes marginal.

The "elbow" of the curve represents the point at which adding additional clusters provides limited improvement in cluster compactness and therefore serves as an objective estimate of the optimal number of neighbourhood vulnerability profiles.

```
In [39]: # Only numeric principal components
pca_features = pca_df[["PC1", "PC2", "PC3"]]

wcsc = []

cluster_range = range(2, 11)

for k in cluster_range:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=20    )

    kmeans.fit(pca_features)

    wcsc.append(kmeans.inertia_)
```

```
In [40]: # Elbow Method
```

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Use only the numeric PCA scores
pca_features = pca_df[["PC1", "PC2", "PC3"]]

wcss = []

cluster_range = range(2, 11)

for k in cluster_range:

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=20    )

    kmeans.fit(pca_features)

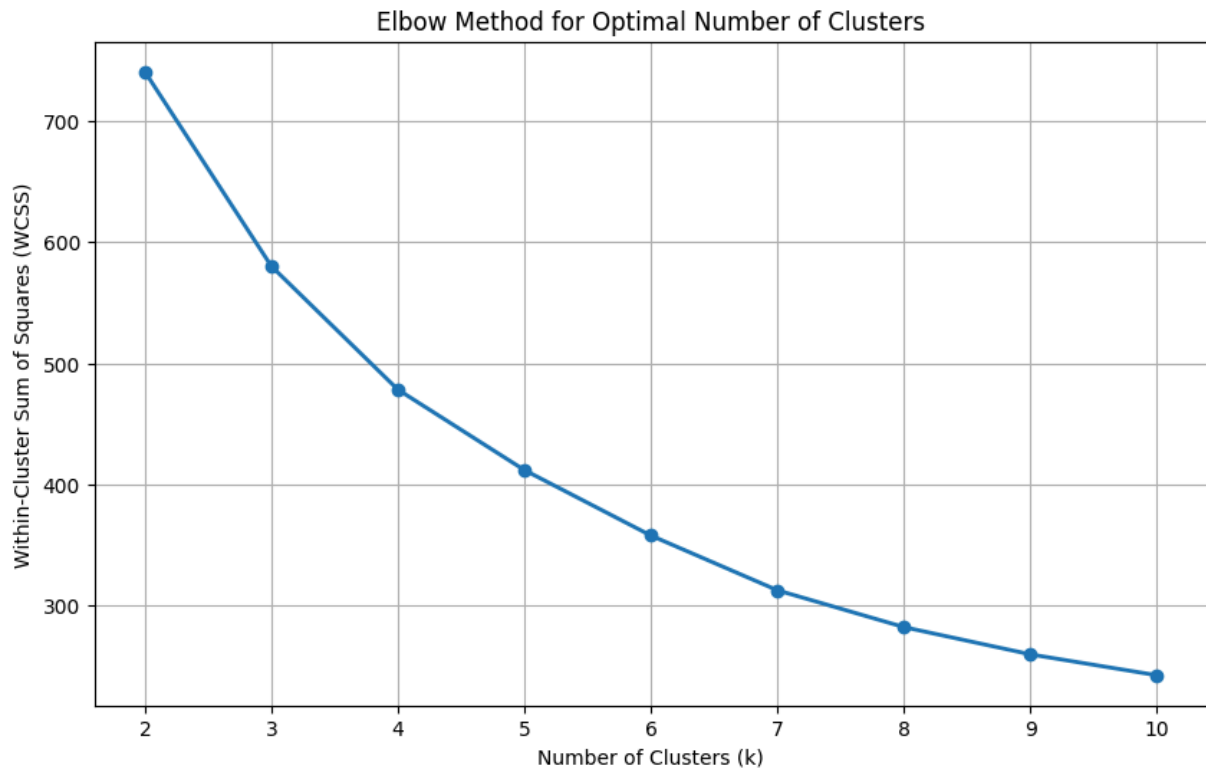
    wcss.append(kmeans.inertia_)

# Plot Elbow Curve
plt.figure(figsize=(10,6))

plt.plot(
    cluster_range,
    wcss,
    marker='o',
    linewidth=2)

plt.xticks(cluster_range)
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Within-Cluster Sum of Squares (WCSS)")
plt.title("Elbow Method for Optimal Number of Clusters")
plt.grid(True)

plt.show()
```



The graph presents the Elbow Method used to determine the optimal number of clusters for K-Means clustering. As the number of clusters (K) increases, the Within-Cluster Sum of Squares (WCSS) decreases because observations are grouped into smaller, more homogeneous clusters. The curve shows a noticeable change in slope around $K = 4$, after which additional clusters result in only modest reductions in WCSS. This "elbow" suggests that four clusters provide a reasonable balance between minimizing within-cluster variation and avoiding an unnecessarily large number of clusters.

8.3 Silhouette Analysis

The Silhouette Score measures how well each neighbourhood fits within its assigned cluster compared with neighbouring clusters.

Silhouette values range from **-1** to **1**, where:

- values close to **1** indicate well-separated clusters;
- values close to **0** indicate overlapping clusters; and
- negative values suggest that observations may have been assigned to an inappropriate cluster.

The clustering solution with the highest average silhouette score is generally considered the most appropriate representation of the underlying neighbourhood vulnerability structure.

```
In [41]: # Silhouette Scores  
  
from sklearn.metrics import silhouette_score
```

```
from sklearn.cluster import KMeans

# Use only the numeric PCA scores
pca_features = pca_df[["PC1", "PC2", "PC3"]]

silhouette_results = []

for k in range(2, 11):

    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=20    )

    labels = kmeans.fit_predict(pca_features)

    score = silhouette_score(
        pca_features,
        labels    )

    silhouette_results.append([k, score])

silhouette_df = pd.DataFrame(
    silhouette_results,
    columns=[
        "Clusters",
        "Silhouette Score"    ])

display(silhouette_df.round(3))
```

	Clusters	Silhouette Score
0	2	0.406
1	3	0.348
2	4	0.302
3	5	0.302
4	6	0.283
5	7	0.276
6	8	0.301
7	9	0.284
8	10	0.264

Interpretation

The Elbow Method demonstrated a substantial reduction in the Within-Cluster Sum of Squares (WCSS) as the number of clusters increased, with the rate of improvement beginning to level off around four clusters. This suggests that a four-cluster solution

provides a reasonable balance between minimizing within-cluster variation and maintaining an interpretable clustering structure.

To further evaluate cluster quality, the Silhouette Score was calculated for clustering solutions ranging from two to ten clusters. The highest average Silhouette Score (0.406) was obtained when **K = 2**, indicating that the two-cluster solution produced the strongest separation between clusters and the highest within-cluster cohesion.

Although the Silhouette Analysis favors two clusters, the Elbow Method suggests that additional clusters provide greater analytical detail while still maintaining acceptable cluster compactness. Since the objective of this study is to identify meaningful socioeconomic neighbourhood profiles rather than simply maximize the Silhouette Score, a four-cluster solution was selected for the final K-Means analysis.

Consequently, **four neighbourhood socioeconomic clusters** were retained for the subsequent clustering analysis.

9. K-Means Clustering

Determination of the Optimal Number of Clusters

The Elbow Method and Silhouette Analysis were used to determine an appropriate number of clusters for the K-Means algorithm.

The Elbow Method showed a substantial reduction in the Within-Cluster Sum of Squares (WCSS) as the number of clusters increased. The rate of improvement began to decrease around 4 clusters, suggesting that additional clusters beyond this point provide only limited improvements in cluster compactness.

The Silhouette Analysis evaluated the quality of the clustering solutions by measuring both within-cluster cohesion and between-cluster separation. The highest average Silhouette Score (0.406) was obtained for the 2-cluster solution, indicating that 2 clusters produce the strongest overall separation among the solutions evaluated.

Although the Silhouette Analysis favors 2 clusters, the Elbow Method indicates that 4 clusters provide a better balance between reducing within-cluster variation and capturing meaningful differences among Toronto neighbourhoods. A 2-cluster solution would primarily separate neighbourhoods into broad socioeconomic categories, whereas a 4-cluster solution allows for the identification of more detailed socioeconomic neighbourhood profiles.

Therefore, **four clusters (K = 4)** were selected for the final K-Means clustering analysis, as this solution provides greater analytical value while maintaining acceptable clustering performance.

```
In [42]: # Final K-Means Clustering

from sklearn.cluster import KMeans

# Use only the principal component scores
pca_features = pca_df[["PC1", "PC2", "PC3"]]

# Fit final K-Means model
kmeans = KMeans(
    n_clusters=4,
    random_state=42,
    n_init=20)

# Assign clusters
pca_df["Cluster"] = kmeans.fit_predict(pca_features)

print("=" * 60)
print("FINAL K-MEANS CLUSTERING COMPLETE")
print("=" * 60)

display(pca_df.head(10))
```

```
=====
FINAL K-MEANS CLUSTERING COMPLETE
=====
```

	Neighbourhood	PC1	PC2	PC3	Cluster
0	Agincourt North	-3.653043	-0.220271	1.660726	3
1	Agincourt South-Malvern West	-2.829893	-0.057081	0.786125	3
2	Alderwood	1.167067	-2.895666	0.585704	0
3	Annex	3.150354	2.456263	0.228375	2
4	Banbury-Don Mills	0.040659	0.326411	1.213919	0
5	Bathurst Manor	0.304375	-0.409320	-0.149427	0
6	Bay Street Corridor	-0.111897	3.350512	-1.582590	2
7	Bayview Village	-0.112480	1.361990	0.136829	2
8	Bayview Woods-Steeles	-0.968461	-0.646169	-0.148910	0
9	Bedford Park-Nortown	2.403369	-1.106511	1.500695	1

9.1 Cluster sizes

```
In [43]: # Cluster Sizes

cluster_sizes = (
    pca_df["Cluster"]
    .value_counts()
    .sort_index()
    .rename_axis("Cluster")
)
```

```
.reset_index(name="Neighbourhoods"))

cluster_sizes["Percentage"] = (
    cluster_sizes["Neighbourhoods"] / len(pca_df) * 100).round(1)

display(cluster_sizes)
```

	Cluster	Neighbourhoods	Percentage
0	0	35	25.0
1	1	43	30.7
2	2	16	11.4
3	3	46	32.9

9.2- Cluster Visualization

```
In [44]: # Cluster Visualization

import matplotlib.pyplot as plt

plt.figure(figsize=(10, 7))

# Plot each cluster separately
for cluster in sorted(pca_df["Cluster"].unique()):

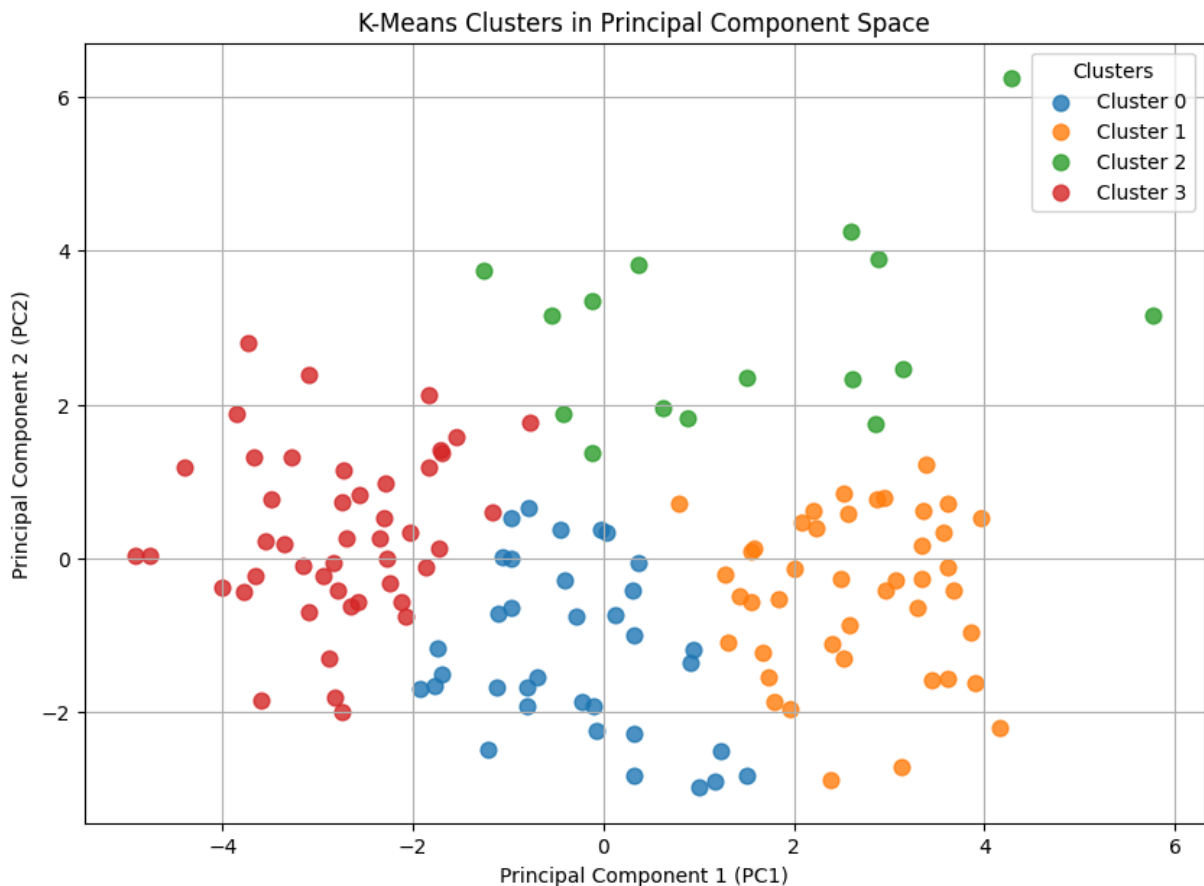
    cluster_data = pca_df[pca_df["Cluster"] == cluster]

    plt.scatter(
        cluster_data["PC1"],
        cluster_data["PC2"],
        s=60,
        alpha=0.8,
        label=f"Cluster {cluster}"    )

plt.xlabel("Principal Component 1 (PC1)")
plt.ylabel("Principal Component 2 (PC2)")
plt.title("K-Means Clusters in Principal Component Space")

plt.legend(title="Clusters")
plt.grid(True)

plt.show()
```



Interpretation

The plot presents the 4 clusters identified by the K-Means algorithm using the first two principal components. Each point represents one Toronto neighbourhood, while the colour indicates the cluster assigned by the algorithm.

The visualization demonstrates that the neighbourhoods are grouped into four distinct regions of the principal component space. Cluster 3 is primarily located on the negative side of Principal Component 1, whereas Cluster 1 occupies the positive side of Principal Component 1. Cluster 2 is concentrated in the upper region of the plot, corresponding to higher Principal Component 2 values, while Cluster 0 is mainly distributed in the lower-central region.

Although some overlap exists between neighbouring clusters, the overall separation indicates that the K-Means algorithm successfully identified groups of neighbourhoods with similar socioeconomic patterns based on the retained principal components. The greater vertical spread observed for Cluster 2 suggests higher variability among the neighbourhoods assigned to this cluster, whereas Clusters 0 and 3 appear comparatively more compact.

The scatter plot provides a visual confirmation of the clustering results. However, the socioeconomic meaning of each cluster cannot be determined from this figure alone. The interpretation of the clusters requires examination of the original socioeconomic indicators, which is presented in the following section.

9.3- Cluster Socioeconomic Profiles

To interpret the neighbourhood clusters, descriptive statistics are calculated for each cluster using the original socioeconomic indicators.

The cluster profiles quantify the average socioeconomic characteristics associated with each vulnerability profile, allowing meaningful comparisons across neighbourhood groups.

These summary statistics provide the evidence required to determine whether the identified clusters represent statistically distinct neighbourhood vulnerability profiles.

```
In [45]: # =====
# Variables for Cluster Profile Interpretation
# =====

profile_variables = [
    "Participation_Rate",
    "Employment_Rate",
    "Unemployment_Rate",
    "Bachelors_Degree_Pct",
    "No_Diploma_Pct",
    "Immigrant_Population_Pct",
    "Visible_Minority_Pct",
    "Renters_Pct",
    "Core_Housing_Need",
    "Unaffordable_Housing",
    "Lone_Parent_Families_Pct"]

print("Cluster profile variables loaded.")
print(f"Number of variables: {len(profile_variables)}")
```

Cluster profile variables loaded.

Number of variables: 11

```
In [46]: kmeans = KMeans(n_clusters=4, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
df["Cluster"] = clusters
```

```
In [47]: cluster_profiles = (
    df
    .groupby("Cluster")[profile_variables]
    .mean()
    .round(2))

display(cluster_profiles)
```

Cluster	Participation_Rate	Employment_Rate	Unemployment_Rate	Bachelors_Degree_Pct	No
0	61.81	56.24	9.00	21.95	
1	68.01	63.40	6.80	22.45	
2	74.36	69.38	6.78	28.06	
3	59.64	53.69	10.00	12.26	

In [48]: *# Cluster Socioeconomic Profiles*

```

profile_variables = [
    "Participation_Rate",
    "Employment_Rate",
    "Unemployment_Rate",
    "Bachelors_Degree_Pct",
    "No_Diploma_Pct",
    "Immigrant_Population_Pct",
    "Visible_Minority_Pct",
    "Renters_Pct",
    "Unaffordable_Housing",
    "Lone_Parent_Families_Pct"]

cluster_profiles = (
    df
    .groupby("Cluster")[profile_variables]
    .mean()
    .round(2))

# Transpose for easier comparison
cluster_profiles = cluster_profiles.T

cluster_profiles.columns = [
    "Cluster 0",
    "Cluster 1",
    "Cluster 2",
    "Cluster 3"]

display(cluster_profiles)

```

	Cluster 0	Cluster 1	Cluster 2	Cluster 3
Participation_Rate	61.81	68.01	74.36	59.64
Employment_Rate	56.24	63.40	69.38	53.69
Unemployment_Rate	9.00	6.80	6.78	10.00
Bachelors_Degree_Pct	21.95	22.45	28.06	12.26
No_Diploma_Pct	19.86	17.84	16.62	24.18
Immigrant_Population_Pct	54.80	31.85	35.54	55.05
Visible_Minority_Pct	58.51	26.08	34.12	66.35
Renters_Pct	22.93	17.31	32.82	15.66
Unaffordable_Housing	42.55	30.51	40.30	35.40
Lone_Parent_Families_Pct	5.31	4.56	3.87	7.27

```
In [49]: # Cluster Labels

cluster_labels = pd.DataFrame({
    "Cluster": ["Cluster 0", "Cluster 1", "Cluster 2", "Cluster 3"],
    "Socioeconomic Profile": [
        "Moderate Vulnerability",
        "Low Vulnerability",
        "Housing Pressure",
        "High Vulnerability"
    ]})

display(cluster_labels)
```

	Cluster	Socioeconomic Profile
0	Cluster 0	Moderate Vulnerability
1	Cluster 1	Low Vulnerability
2	Cluster 2	Housing Pressure
3	Cluster 3	High Vulnerability

The cluster profile analysis highlights meaningful socioeconomic differences among the 4 neighbourhood groups. Cluster 1 demonstrates the strongest overall socioeconomic conditions, with high employment, high educational attainment, and low unemployment. Cluster 2 also has strong labour market performance but faces greater housing affordability challenges and higher low-income prevalence. Cluster 0 represents neighbourhoods with generally moderate socioeconomic characteristics. In contrast, Cluster 3 exhibits the greatest socioeconomic vulnerability, characterized by lower employment, higher unemployment, lower educational attainment, and the highest low-income levels. Overall, these results show that the K-Means algorithm successfully identified neighbourhoods with distinct socioeconomic profiles, providing valuable insights for policy and community planning.

Interpretation

The cluster profiling analysis demonstrates that the four clusters identified through K-Means represent distinct neighbourhood socioeconomic profiles across Toronto.

Cluster 0 (Moderate Vulnerability) consists of neighbourhoods with moderate employment, education, and housing indicators. These neighbourhoods exhibit relatively balanced socioeconomic characteristics and lower levels of low-income prevalence compared with the more vulnerable clusters.

Cluster 1 (Low Vulnerability) represents the strongest socioeconomic profile. This cluster is characterized by high employment and labour force participation, low unemployment, higher educational attainment, and the lowest low-income prevalence, indicating relatively favourable socioeconomic conditions.

Cluster 2 (Housing Pressure) includes neighbourhoods with the highest employment and educational attainment but also the highest proportions of renters and unaffordable housing. Despite strong labour market outcomes, these neighbourhoods continue to experience significant housing affordability pressures, suggesting that housing costs remain a challenge even in economically strong areas.

Cluster 3 (High Vulnerability) represents the most socioeconomically vulnerable neighbourhoods. This cluster is characterized by lower employment and participation rates, higher unemployment, lower educational attainment, higher proportions of immigrant and visible minority populations, and the highest low-income prevalence, indicating neighbourhoods that may require greater policy attention and targeted community support.

Overall, the clustering results demonstrate that Toronto neighbourhoods can be grouped into distinct socioeconomic profiles based on employment, education, housing, demographic, and income characteristics. These findings provide valuable insights for identifying neighbourhoods with different levels of socioeconomic vulnerability and can support evidence-based planning and resource allocation.

9.4. Cluster Profile Heatmap

To facilitate comparison across neighbourhood vulnerability profiles, a heatmap is constructed using the average socioeconomic characteristics of each cluster.

The visualization highlights the relative strengths and weaknesses of each neighbourhood profile and complements the numerical summary presented in the cluster profile table.

```
In [50]: import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))
```

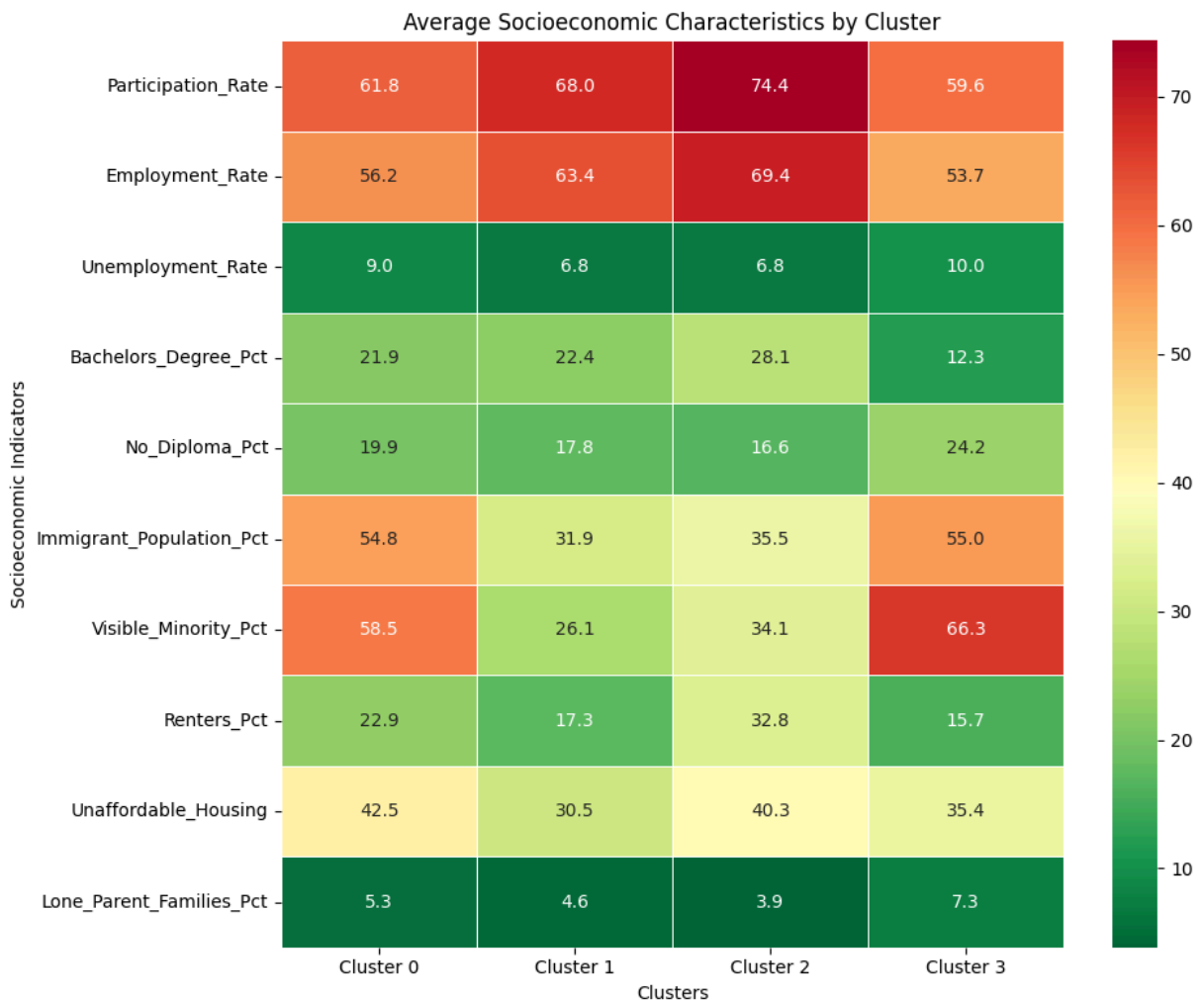
```

sns.heatmap(
    cluster_profiles,
    annot=True,
    fmt=".1f",
    cmap="RdYlGn_r",
    linewidths=0.5)

plt.title("Average Socioeconomic Characteristics by Cluster")
plt.xlabel("Clusters")
plt.ylabel("Socioeconomic Indicators")

plt.tight_layout()
plt.show()

```



The heatmap provides a visual comparison of the average socioeconomic characteristics across the four neighbourhood clusters. Clear differences are observed among the clusters, confirming that the K-Means algorithm successfully identified neighbourhoods with distinct socioeconomic profiles. Cluster 1 demonstrates the strongest socioeconomic conditions, with high employment and participation rates, higher educational attainment, and the lowest unemployment and low-income prevalence. Cluster 2 also exhibits strong labour market outcomes but stands out for its higher levels of renters and housing unaffordability, indicating greater housing affordability pressures. Cluster 0 represents neighbourhoods with

moderate socioeconomic characteristics across most indicators. In contrast, Cluster 3 displays the greatest socioeconomic vulnerability, characterized by lower employment and participation rates, higher unemployment, lower educational attainment, larger immigrant and visible minority populations, and the highest low-income prevalence. Overall, the heatmap reinforces the findings from the cluster profile analysis by visually highlighting the socioeconomic differences among the neighbourhood groups.

10. Statistical Validation of Clusters Using One-Way ANOVA

Objective

Principal Component Analysis (PCA) and K-Means clustering identified groups of Toronto neighbourhoods with similar socioeconomic characteristics. However, clustering algorithms alone do not determine whether these groups differ statistically.

To formally evaluate the validity of the identified clusters, a **One-Way Analysis of Variance (ANOVA)** is performed.

The analysis investigates whether the dependent variable, average **Low Income After-Tax Percentage (LIM-AT)**, which is used in this study as a **proxy measure of neighbourhood socioeconomic vulnerability**, differs significantly across the identified neighbourhood clusters.

Demonstrating statistically significant differences between clusters strengthens the evidence that the clustering process successfully identified distinct socioeconomic neighbourhood profiles.

Supervised statistical Test- ANOVA (one way)

10.1 Research Hypotheses

The One-Way ANOVA evaluates whether the mean neighbourhood Low Income After-Tax Percentage (LIM-AT) differs across the identified K-Means clusters.

Null Hypothesis (H_0)

The mean Low Income After-Tax Percentage (LIM-AT) is equal across all neighbourhood clusters.

Alternative Hypothesis (H_1)

At least one neighbourhood cluster has a significantly different mean Low Income After-Tax Percentage (LIM-AT).

10.2 Assumptions of One-Way ANOVA

The validity of the One-Way ANOVA relies on the following assumptions:

- Observations are independent.
- The dependent variable is approximately normally distributed within each cluster.
- Variances are reasonably homogeneous across clusters.

Because the analysis is performed using neighbourhood-level observations and the identified clusters contain multiple observations, the One-Way ANOVA provides an appropriate statistical framework for evaluating whether the clusters differ significantly with respect to the dependent variable.

```
In [51]: # Add the dependent variable back to pca_df
pca_df["Low_Income_AfterTax_Pct"] = df["Low_Income_AfterTax_Pct"].values
```

```
In [52]: from scipy.stats import f_oneway
import pandas as pd

cluster_groups = [
    group["Low_Income_AfterTax_Pct"].values
    for _, group in pca_df.groupby("Cluster")]

f_statistic, p_value = f_oneway(*cluster_groups)

decision = "Reject H0" if p_value < 0.05 else "Fail to Reject H0"

anova_summary = pd.DataFrame({
    "Statistic": [
        "Number of Clusters",
        "F-statistic",
        "p-value",
        "Decision"    ],
    "Value": [
        len(cluster_groups),
        round(f_statistic, 3),
        "< 0.001" if p_value < 0.001 else round(p_value, 3),
        decision    ]})

display(anova_summary)
```

	Statistic	Value
0	Number of Clusters	4
1	F-statistic	32.58
2	p-value	< 0.001
3	Decision	Reject H ₀

```
In [53]: cluster_means = (
    pca_df.groupby("Cluster")["Low_Income_AfterTax_Pct"]
        .mean()
        .round(2)
```

```
.reset_index(name="Mean Low Income After-Tax (%)")\n\ndisplay(cluster_means)
```

	Cluster	Mean Low Income After-Tax (%)
0	0	16.74
1	1	13.77
2	2	24.17
3	3	25.38

Interpretation

A one-way ANOVA was conducted to determine whether the mean Low Income After-Tax Percentage (LIM-AT) differed across the four neighbourhood clusters identified using K-Means clustering. The analysis produced an F-statistic of 32.580 with a p-value < 0.001, indicating statistically significant differences in the mean LIM-AT values among the clusters. Accordingly, the null hypothesis (H_0) was rejected, confirming that at least one cluster has a significantly different average low-income prevalence.

These findings provide strong statistical support for the K-Means clustering solution, demonstrating that the identified neighbourhood clusters represent distinct socioeconomic vulnerability profiles rather than random groupings. The significant differences in LIM-AT across the clusters validate the effectiveness of the clustering approach in distinguishing neighbourhoods with varying levels of socioeconomic vulnerability.

11. Conclusion

Research Question 2

Can Toronto neighbourhoods be grouped into statistically distinct socioeconomic vulnerability profiles based on selected socioeconomic indicators?

Yes.

The analyses performed in this notebook provide strong statistical evidence that Toronto neighbourhoods can be grouped into distinct socioeconomic profiles based on selected socioeconomic indicators.

Principal Component Analysis (PCA) successfully reduced the original 11 independent socioeconomic indicators to three principal components, retaining 84.61% of the total variation while substantially reducing redundancy among highly correlated variables. As described earlier, PCA was used solely as a dimensionality reduction technique to improve clustering performance and was not used for predictive modelling. The proxy outcome variable, Low Income After-Tax Percentage (LIM-AT), was intentionally excluded from the PCA

and K-Means clustering process to ensure that the clusters were formed using only the independent socioeconomic indicators.

Cluster validation using the Elbow Method and Silhouette Analysis supported the selection of a four-cluster solution, providing an appropriate balance between within-cluster similarity and between-cluster separation.

The resulting neighbourhood clusters represent distinct socioeconomic profiles:

Cluster 0 (35 neighbourhoods, 25.0%) – Moderate Socioeconomic Vulnerability:

Characterized by moderate employment and education levels with intermediate levels of housing need and low-income prevalence. Cluster 1 (43 neighbourhoods, 30.7%) – Low

Socioeconomic Vulnerability: Characterized by the lowest average Low Income After-Tax Percentage (LIM-AT), strong labour market performance, higher educational attainment, and comparatively favourable housing conditions. Cluster 2 (16 neighbourhoods, 11.4%) –

Housing Pressure Profile: Characterized by relatively high employment and educational attainment but elevated housing affordability pressures and renter concentrations. Cluster 3

(46 neighbourhoods, 32.9%) – High Socioeconomic Vulnerability: Characterized by the highest average Low Income After-Tax Percentage (LIM-AT), lower employment and educational attainment, and greater socioeconomic disadvantage.

The clustering solution was statistically validated using a One-Way Analysis of Variance (ANOVA). The analysis demonstrated significant differences in the mean Low Income After-Tax Percentage (LIM-AT) among the four neighbourhood clusters ($F = 32.580$, $p < 0.001$).

Since LIM-AT was not used to generate the clusters, these findings provide strong evidence that neighbourhoods grouped solely according to socioeconomic characteristics also exhibit significantly different levels of low-income prevalence.

Overall, the findings demonstrate that Toronto neighbourhoods can be grouped into statistically distinct socioeconomic vulnerability profiles using independent socioeconomic indicators. The identified clusters provide an interpretable and statistically validated framework for understanding neighbourhood socioeconomic conditions and may assist municipal planners, policymakers, and community organizations in identifying areas where targeted socioeconomic interventions and resource allocation may be most beneficial.

```
In [54]: # Export Final Clustered Dataset

output_path = (
    project_root
    / "data"
    / "processed"
    / "toronto_neighbourhood_2016_clusters.csv")

df.to_csv(
    output_path,
    index=False)
```

```
print("=" * 60)
print("CLUSTER DATASET EXPORTED")
print("=" * 60)
print(output_path)
```

```
=====
CLUSTER DATASET EXPORTED
=====
c:\Users\ksalyamova\Desktop\CIND 820\CIND820-Capstone-Project\data\processed\toronto
_neighbourhood_2016_clusters.csv
```